

# LABORATORY ASSIGNMENT

**Course:** Full Stack Web Development  
**Course Code:** 23CM4121  
**Assignment Topic:** JavaScript Inheritance Types (ES6)  
**Student Name:** A. Bhanu Prasad  
**Roll No.:** A24126552068  
**Date:** 30-08-2026

---

## 1. TITLE

Implementation of JavaScript ES6 Inheritance Types (Single, Multilevel, Hierarchical, Multiple, and Hybrid)

## 2. AIM

To design and implement JavaScript ES6 class programs demonstrating Single Inheritance, Multilevel Inheritance, Hierarchical Inheritance, Multiple Inheritance (via prototype mixing), and Hybrid Inheritance using the class and extends keywords.

## 3. OBJECTIVE

- To understand Object-Oriented Programming (OOP) concepts in modern JavaScript (ES6+).
- To implement Single Inheritance where a child class extends a single parent class.
- To implement Multilevel Inheritance where a derived class inherits from another derived class forming a chain.
- To implement Hierarchical Inheritance where multiple child classes inherit from a single parent class.
- To implement Multiple Inheritance in JavaScript using prototype composition.
- To implement Hybrid Inheritance combining multilevel and hierarchical inheritance structures.

## 4. THEORY

Inheritance is a fundamental concept in Object-Oriented Programming that allows a class (child/subclass) to acquire properties and methods from another class (parent/superclass). ES6 introduced standard class and extends syntax for inheritance in JavaScript.

1. Single Inheritance: A child class inherits directly from one parent class.
2. Multilevel Inheritance: A class extends a parent class which itself extends another base class (Class A -> Class B -> Class C).
3. Hierarchical Inheritance: Multiple child classes extend the same base class (Class A -> Class B, Class A -> Class C).
4. Multiple Inheritance: A class inherits methods from multiple parent classes (achieved via prototype mixing in JS).
5. Hybrid Inheritance: A combination of two or more types of inheritance (e.g. Multilevel + Hierarchical).

## 5. PROGRAM IMPLEMENTATIONS

### 5.1 Single Inheritance (SingleInheritance.js)

```
class Animal{
    eat(){
        console.log("Animal is eating");
    }
}
class Dog extends Animal{
    bark(){
        console.log("Dog is barking");
    }
}
let d=new Dog();
d.eat();
d.bark();
```

### 5.2 Multilevel Inheritance (MultilevelInheritance.js)

```
class Animal{
    eat(){
        console.log("Animal is eating");
    }
}
class Dog extends Animal{
    bark(){
        console.log("Dog is barking");
    }
}
class Puppy extends Dog{
    play(){
        console.log("Puppy is playing");
    }
}
let p=new Puppy();
p.eat();
p.bark();
p.play();
```

### 5.3 Hierarchical Inheritance (Hierarchical\_Inheritance.js)

```
class Animal{
    eat(){
        console.log("Animal is eating");
    }
}
class Dog extends Animal{
    bark(){
        console.log("Dog is barking");
    }
}
class Cat extends Animal{
    meow(){
        console.log("Cat is meowing");
    }
}
let d=new Dog();
d.eat();
d.bark();
let c=new Cat();
c.eat();
c.meow();
```

## 5.4 Multiple Inheritance (Multiple\_Inheritance.js)

```
class Father{
  drive(){
    console.log("Father can drive");
  }
}
class Mother{
  cook(){
    console.log("Mother can cook");
  }
}
class Child extends Father{
  code(){
    console.log("Child can code");
  }
}

Child.prototype.cook=Mother.prototype.cook;

let c=new Child();

c.drive();
c.cook();
c.code();
```

## 5.5 Hybrid Inheritance (Hybrid\_Inheritance.js)

```
class Animal{
  eat(){
    console.log("Animal is eating");
  }
}
class Dog extends Animal{
  bark(){
    console.log("Dog is barking");
  }
}
class Puppy extends Dog{
  play(){
    console.log("Puppy is playing");
  }
}
class Cat extends Animal{
  meow(){
    console.log("Cat is meowing");
  }
}

let p=new Puppy();
p.eat();
p.bark();
p.play();
let c=new Cat();

c.eat();
c.meow();
```

## 6. EXECUTION OUTPUT SCREENSHOTS

### Output 1: Single Inheritance Execution

```
JS SingleInheritance.js > ...
1  class Animal{
2    |   eat(){
3    |     |   console.log("Animal is eating");
4    |     }
5    |   }
6  class Dog extends Animal{
7    |   bark(){
8    |     |   console.log("Dog is barking");
9    |     }
10   |   }
11   let d=new Dog();
12   d.eat();
13   d.bark();

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\bhanu\OneDrive\Desktop\fullstack> node SingleInheritance.js
Animal is eating
Dog is barking
PS C:\Users\bhanu\OneDrive\Desktop\fullstack> █
```

### Output 2: Multilevel Inheritance Execution

```
JS MultilevelInheritance.js > ...
1  class Animal{
2    |   eat(){
3    |     |   console.log("Animal is eating");
4    |     }
5    |   }
6  class Dog extends Animal{
7    |   bark(){
8    |     |   console.log("Dog is barking");
9    |     }
10   |   }
11  class Puppy extends Dog{
12    |   play(){
13    |     |   console.log("Puppy is playing");
14    |     }
15   |   }
16   let p=new Puppy();
17   p.eat();
18   p.bark();
19   p.play();

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\bhanu\OneDrive\Desktop\fullstack> node MultilevelInheritance.js
Animal is eating
Dog is barking
Puppy is playing
PS C:\Users\bhanu\OneDrive\Desktop\fullstack> █
```

### Output 3: Hierarchical Inheritance Execution

```
JS Hierarchical_Inheritance.js > ...
1  class Animal{
2      eat(){
3          console.log("Animal is eating");
4      }
5  }
6  class Dog extends Animal{
7      bark(){
8          console.log("Dog is barking");
9      }
10 }
11 class Cat extends Animal{
12     meow(){
13         console.log("Cat is meowing");
14     }
15 }
16 let d=new Dog();
17 d.eat();
18 d.bark();
19 let c=new Cat();
20 c.eat();
21 c.meow();
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\bhanu\OneDrive\Desktop\fullstack> node Hierarchical_Inheritance.js
Animal is eating
Dog is barking
Animal is eating
Cat is meowing
PS C:\Users\bhanu\OneDrive\Desktop\fullstack>
```

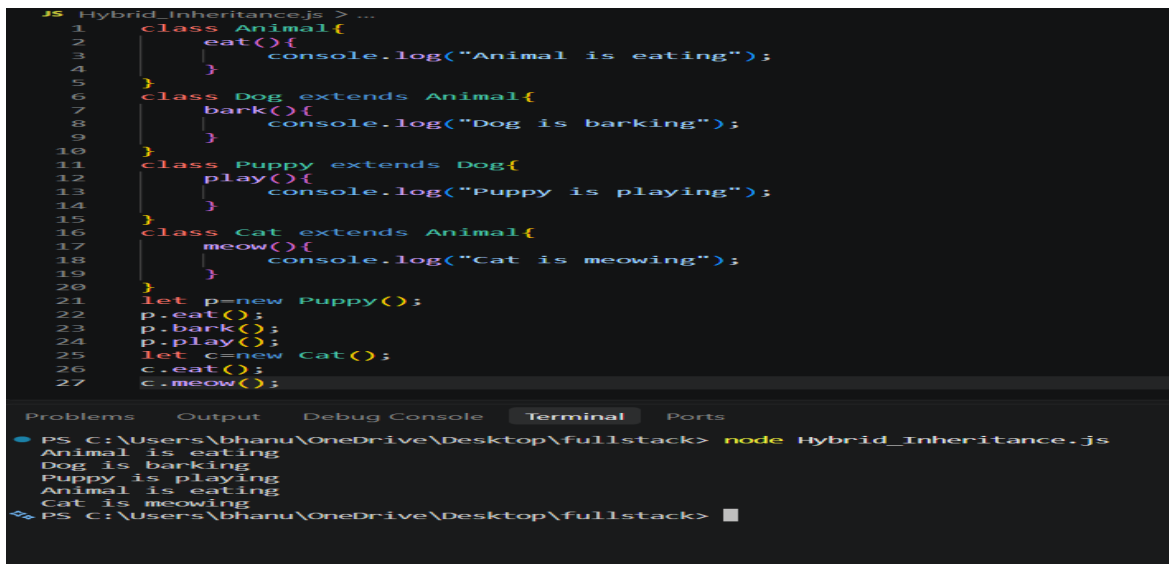
### Output 4: Multiple Inheritance Execution

```
JS Multiple_Inheritance.js > ...
1  class Father{
2      drive(){
3          console.log("Father can drive");
4      }
5  }
6  class Mother{
7      cook(){
8          console.log("Mother can cook");
9      }
10 }
11 class Child extends Father{
12     code(){
13         console.log("Child can code");
14     }
15 }
16
17 Child.prototype.cook=Mother.prototype.cook;
18
19 let c=new Child();
20
21 c.drive();
22 c.cook();
23 c.code();
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\bhanu\OneDrive\Desktop\fullstack> node Multiple_Inheritance.js
Father can drive
Mother can cook
Child can code
PS C:\Users\bhanu\OneDrive\Desktop\fullstack>
```

### Output 5: Hybrid Inheritance Execution



```
JS Hybrid_Inheritance.js > ...
1  class Animal{
2      eat(){
3          console.log("Animal is eating");
4      }
5  }
6  class Dog extends Animal{
7      bark(){
8          console.log("Dog is barking");
9      }
10 }
11 class Puppy extends Dog{
12     play(){
13         console.log("Puppy is playing");
14     }
15 }
16 class Cat extends Animal{
17     meow(){
18         console.log("Cat is meowing");
19     }
20 }
21 let p=new Puppy();
22 p.eat();
23 p.bark();
24 p.play();
25 let c=new Cat();
26 c.eat();
27 c.meow();
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\bhanu\OneDrive\Desktop\fullstack> node Hybrid_Inheritance.js
Animal is eating
Dog is barking
Puppy is playing
Animal is eating
Cat is meowing
PS C:\Users\bhanu\OneDrive\Desktop\fullstack>
```

## 7. RESULT

Thus, all five types of JavaScript inheritance (Single, Multilevel, Hierarchical, Multiple, and Hybrid) were successfully implemented using ES6 classes and verified through Node.js execution.